```
satwika@SATWIKAG:~$ nano gcc_program1.c
satwika@SATWIKAG:~$ gcc gcc_program1.c -o gcc_program1
satwika@SATWIKAG:~$ ./gcc_program1
PID = 918
satwika@SATWIKAG:~$ nano gcc_program2.c
satwika@SATWIKAG:~$ gcc gcc_program2.c -o gcc_program2
satwika@SATWIKAG:~$ ./gcc_program2
Hello
Hello
satwika@SATWIKAG:~$ nano gcc_program3.c
satwika@SATWIKAG:~$ gcc gcc_program3.c -o gcc_program3
satwika@SATWIKAG:~$ ./gcc_program3
I am Parent
I am Child
satwika@SATWIKAG:~$ nano gcc_program4.c
satwika@SATWIKAG:~$ gcc gcc_program4.c -o gcc_program4
satwika@SATWIKAG:~$ ./gcc_program4
Parent PID = 1028
Child PID = 1029
Child PID = 1029
Parent PID = 332
satwika@SATWIKAG:~$ nano gcc_program5.c
satwika@SATWIKAG:~$ gcc gcc_program5.c -o gcc_program5
satwika@SATWIKAG:~$ ./gcc_program5
Child Executing
Parent Executing
satwika@SATWIKAG:~$ nano gcc_program6.c
satwika@SATWIKAG:~$ gcc gcc_program6.c -o gcc_program6
satwika@SATWIKAG:~$ ./gcc_program6
PID = 1098
PID = 1100
PID = 1099
PID = 1101
```

fork()

```
=== gcc_program1.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
printf("PID = %d\n", getpid());
return 0;
}
```

```
=== gcc_program2.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
fork();
printf("Hello\n");
return 0;
}
--------------------------------
```

```
=== gcc_program3.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid;
pid = fork();
if(pid == 0)
printf("I am Child\n");
else
printf("I am Parent\n");
return 0;
}
```

```
=== gcc_program3.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid;
pid = fork();
if(pid == 0)
printf("I am Child\n");
else
printf("I am Parent\n");
return 0;
}
```

```
=== gcc_program5.c ===
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
pid_t pid = fork();
if(pid == 0)
{
printf("Child Executing\n");
}
else
{
wait(NULL);
printf("Parent Executing\n");
}
return 0;
}
```

```
=== gcc_program6.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
fork();
fork();
printf("PID = %d\n", getpid());
return 0;
}
```

```
=== gcc_program7.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid = fork();
if(pid == 0)
{
printf("Child executing ls...\n");
execl("/bin/ls","ls","-l",NULL);
}
else
{
printf("Parent Waiting...\n");
}
return 0;
}
```

```
=== gcc_program10.c ===
#include <stdio.h>
#include <unistd.h>
int main()
{
int i;
for(i=0;i<3;i++)
fork();
printf("PID = %d\n",getpid());
return 0;
}
```

```
satwika@SATWIKAG:~$ cat gcc_program10.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char cmd[50];
    char *args[20];

    printf("Enter Linux Command: ");

    if (fgets(cmd, sizeof(cmd), stdin) == NULL) {
        return 1;
    }

    cmd[strcspn(cmd, "\n")] = '\0';

    int i = 0;
    char *token = strtok(cmd, " ");
    while (token != NULL && i < 19) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;

    if (args[0] == NULL) {
        return 0;
    }

    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork Failed\n");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        execvp(args[0], args);

        perror("Command Execution Failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        wait(NULL);
        printf("\nChild Process Completed\n");
    }
    return 0;
}
```

exec()

```
satwika@SATWIKAG:~$ head -n 25 exec[1-5].c
==> exec1.c <==
#include <stdio.h>
#include <unistd.h>
int main()
{
   printf("Before execl()\n");
   execl("/bin/ls", "ls", "-l", NULL);
   printf("This will not print if execl() succeeds.\n");
   return 0;
}

==> exec2.c <==
#include <stdio.h>
#include <unistd.h>
int main()
{
   printf("Before execlp()\n");
   execlp("date", "date", NULL);
   printf("This will not print if execlp() succeeds.\n");
   return 0;
}

==> exec3.c <==
#include <stdio.h>
#include <unistd.h>
int main()
{ char *args[] = {"ls", "-l", NULL};
  printf("Before execv()\n");
  execv("/bin/ls", args);
  printf("This will not print if execv() succeeds.\n");
  return 0;
}
```

```
==> exec4.c <==
#include <stdio.h>
#include <unistd.h>
int main()
{
char *args[] = {"date", NULL};
printf("Before execvp()\n");
execvp("date", args);
printf("This will not print if execvp() succeeds.\n");
return 0;
}


==> exec5.c <==
#include <stdio.h>
#include <unistd.h>

extern char **environ;

int main()
{
    char *args[] = {"ls", "-l", NULL};

    printf("Before execve()\n");
    execve("/bin/ls", args, environ);

    printf("This will not print if execve() succeeds.\n");
    return 0;
}
```